

CLAUDE

Helix Cluster OS — CLAUDE.md

Field	Value
Revision	2
Created	2026-05-30
Last modified	2026-06-02T09:49:46Z
Status	active

INHERITED FROM HelixConstitution/ CLAUDE.md

All rules in HelixConstitution/CLAUDE.md (and the HelixConstitution/Constitution.md it references) apply unconditionally. The project-specific rules below extend them.

Project-Specific Extensions

Technology Stack

- Go 1.24+ for microservices
- Zig 0.14+ for system primitives
- C/C++ for GPU kernels
- PostgreSQL 16+, Redis Cluster 7+, etcd, NATS, Kafka

Architecture

- Seven-layer stack (L0-L7)
- 14 control plane microservices
- SWIM gossip + Raft consensus
- Omega-model scheduler with optimistic concurrency

CLAUDE-1: End-User Usability Guarantee (§7.1 + §11.4.39 Reinforcement)

Forensic anchor — verbatim user mandate: > “We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can’t be used! This **MUST NOT** be the case and execution of tests and Challenges **MUST** guarantee the quality, the completion and full usability by end users of the product!”

Operative rules for AI agents: 1. **Tests are necessary but NOT sufficient.** When implementing or reviewing a feature, you **MUST** verify: - Unit tests exist (with paired mutation per §1.1) - Integration tests against REAL services (not mocks) - End-to-end tests that exercise the feature as an end user would - Challenge validation via HelixQA framework 2. **A test that passes on a non-functional feature is a PASS-bluff** — severity equivalent to §7.1 violation. You **MUST** flag this during code review. 3. **Every test MUST prove the feature works for end users**, not just that code executes without panic. Verify sink-side behavior, not just function return values. 4. **Challenges (HelixQA) are bound equally** — a Challenge PASS on a broken feature is the same class of defect as a unit test PASS on broken code. 5. **No mock-only validation** for features that claim real-world operation. Mocks are permitted **ONLY** in unit tests per §11.4.27. 6. **Sink-side evidence required:** Before declaring a feature complete, you **MUST** require captured evidence (screenshot, log, metrics) proving end-user-visible operation.

CLAUDE-2: Cross-Platform Parity Guarantee

Forensic anchor — verbatim user mandate: > “For any Linux specific technology we **MUST** implement for other platforms (macOS) proper equivalents so on every OS we use proper system equivalents!”

Operative rules for AI agents: 1. **No OS may run on a mock/stub for a feature that claims real operation.** Where a capability is implemented with a Linux-specific mechanism (cgroup-v2, /proc, DRM/sysfs, netlink, kernel WireGuard, namespaces, etc.), you **MUST** implement a *real* equivalent for every supported OS using that OS’s proper system facility (macOS: sysctl/host_statistics64/mach/vm_stat/IOKit/Metal/wireguard-go, etc.). A `_mock.go` / `_other.go` stub behind a `!linux` build tag for a real-operation feature is a CLAUDE-1 PASS-bluff and is **FORBIDDEN**. 2. **Platform code is split by build tags** behind ONE shared interface (e.g. `reader_linux.go` / `reader_darwin.go` implementing `ResourceReader`). Each implementation returns real, measured values for its OS. 3. **Tests prove the feature on the HOST OS — never blanket-skip non-Linux.** An integration test must exercise the real per-OS path and assert against an independent OS oracle (Linux: `free -b, /proc`; macOS: `sysctl -n hw.memsize, vm_stat`). `t.Skip` is permitted **ONLY** for a capability that genuinely has no equivalent on that OS — and that must be justified in writing, not used to hide an unimplemented port. 4. **Known Linux-specific hotspots requiring macOS equivalents:** `pkg/resources` (`proc_mock.go` → `real proc_darwin.go`; `drm_other.go` → `real macOS GPU probe`), `pkg/wireguard` (kernel WG → `wireguard-go` userspace on macOS), any future `/proc/cgroup/netlink/namespace` use. 5. **Applies retroactively and going forward:** new features ship with all-OS equivalents in the same wave; existing `!linux` stubs are tracked as defects to remediate.

CLAUDE-3: Documentation & Materials Continuous-Sync Guarantee (§11.4.106 docs-chain + §11.4.57/59/60/65/44 + Constitution line “No documentation ever can be out of sync with its codebase”)

Forensic anchor — verbatim user mandate: > “while we continuously change the project, by extending it, implementing new features, applying fixes and changes, whatever we do on project or any of its services, components, architecture or anything else **MUST** trigger proper updating of main README document, all project documentation, user guides, manuals, website(s) (if any), diagrams, graphs, schemes, SQL definitions and all other related materials! We **MUST** be sure that proper changes are applied or new materials created so the documentation and related materials are all **ALWAYS** up to date and in sync (and all of its exports)! Connect all this with docs_chain Submodule and make sure that all this is achieved out of the box, never forgotten or skipped and all docs and materials maintained regularly!”

This rule **restates and cites** the already-binding master mandate — it does NOT redefine it. The canonical authority is HelixConstitution/Constitution.md §11.4.106 (**docs-chain-mechanical-sync**) and its anchor family (§11.4.12/45/53/56/57/59/60/65/86/93/95, §12.10, §11.4.44), under the absolute Constitution rule: “**No documentation ever can be out of sync with its codebase.**” Per §11.4.35 inheritance, this project restates + cites it here.

Operative rules for AI agents (binding on every change — code, service, component, architecture, schema, script, or config):

- 1. Every change triggers a documentation pass — no exceptions.** Whenever you extend the project, add a feature, apply a fix, or change architecture/services/components, in the SAME work unit you **MUST** update (or newly author) every affected material: the main README.md, all docs/**, user guides, manuals, website(s) if any, diagrams/graphs/schemes, SQL/schema definitions, and **all their exports** (md → html/pdf/docx). A change that ships code but leaves docs stale is a §11.4.106 / CLAUDE-3 violation and a release blocker.
- 2. docs_chain is the mechanical enforcer — out of the box, never skipped.** Documentation/DB sync **MUST** be driven through the docs_chain engine (the vasic-digital/docs_chain submodule, §11.4.106), with this project’s chains registered as data in .docs_chain/contexts/*.yaml. Every tracked canonical Markdown source **MUST** have its derived exports kept in sync by docs_chain sync; docs_chain verify is the deterministic pre-build/CI sink-side gate. New materials **MUST** be added to a .docs_chain context so they are enforced automatically — never maintained by hand or by retired ad-hoc scripts.
- 3. No escape hatch.** There is no --skip-docs-chain / --ad-hoc-sync-ok / --fake-transform. A both-dirty pair is a surfaced conflict (never a silent merge); an absent transform tool surfaces a typed ToolAbsentError + honest §11.4.3 SKIP-with-reason — never a fake PASS. Captured evidence lands at qa-results/docs_chain/<run-id>/. 4. **Coverage is complete and current.** .docs_chain/contexts/ **MUST** track the main README, the governance docs (CLAUDE/AGENTS/QWEN), the foundation-package catalogue, the changelog, architecture diagrams, and SQL/schema definitions — not only issues/fixed/registry. When you add a new document or material type, register it in the same change.
- 5. Runs regularly, not just at release.** Documentation sync is maintained continuously (every wave / every change), so materials never drift. Treat a stale doc the same as a failing test.
- 6. This is a standing parallel work stream.** Documentation maintenance runs alongside implementation, using subagent-driven development where it helps — it is never deferred “until later.”

Applied Fixes Table

Fix	Date	Commit	Release	Description
CLAUDE-3 docs-sync guarantee	2026-06-02	(this wave)	Unreleased	Added CLAUDE-3 / AGENT-2 / QWEN-2 restating+citing Constitution §11.4.106 docs-chain mandate; extended .docs_chain/contexts/ tracked_docs.yaml to enforce README + CLAUDE/AGENTS/QWEN + FOUNDATION_PACKAGES + CHANGELOG export-sync out of the box.